

No todas las instrucciones se olvidan igual: medición de la retención por tipo de instrucción en asistentes de programación con LLMs

Tomás Pablo Korenblit

Resumen. Los modelos de lenguaje grandes (LLMs) pierden adherencia a las instrucciones a medida que la conversación se extiende. La literatura mide ese fenómeno, pero siempre de forma agregada: informa una caída promedio de cumplimiento agregando todas las instrucciones. Este trabajo aborda una pregunta que esa literatura deja abierta: ¿decaen todas las instrucciones por igual, o algunas se retienen y otras no? Para responder esto, construí un banco de pruebas que planta doce preferencias de código como pedidos del usuario en conversaciones sobre bases de código y mide si el modelo las respeta al final. Sobre esas observaciones ajusté un modelo bayesiano con efectos jerárquicos por tipo de decisión. El resultado principal es que la retención varía mucho entre tipos de instrucción: los efectos del tratamiento cambian en más de un orden de magnitud ($\sigma_\beta = 2,11$). Tres de doce instrucciones se benefician del refuerzo (dos de ellas de forma robusta entre modelos), una empeora cuando se la refuerza y ocho no requieren intervención, de modo que el refuerzo uniforme malgasta la mayor parte del presupuesto. Esa heterogeneidad es lo que un sistema de refuerzo selectivo necesitaría explotar.

Palabras clave: cumplimiento de instrucciones; context rot; inferencia bayesiana; refuerzo selectivo; evaluación de LLMs.

Introducción

Cuando alguien usa un asistente conversacional como ChatGPT o Claude, la experiencia inicial suele ser buena: explica su proyecto, pide que se respeten ciertas reglas y el modelo obedece. Sin embargo, a medida que la conversación se extiende, en algún momento el modelo deja de respetar lo que se le pidió: sigue respondiendo, pero como si

nunca hubiera recibido esas instrucciones. A este fenómeno se lo denomina *context rot*: la pérdida de adherencia a las instrucciones conforme crece el contexto.

No se trata del defecto de un modelo particular; le pasa a todos, y la causa es arquitectónica. Una es la dilución de la atención: para generar cada token, un LLM decide a qué partes del contexto asignarles más peso mediante un mecanismo de atención que es un recurso finito que se reparte entre todos los tokens. Cuando el contexto crece (las ventanas pasaron de decenas de miles de tokens en 2023 a un millón desde principios de 2024, aproximadamente dos mil páginas), las instrucciones del comienzo compiten por atención con todo lo posterior. Otra es el sesgo posicional: la información del medio del contexto pesa menos que la de los extremos (Liu et al., 2024). La degradación, además, no aparece solo cerca del límite de la ventana: se observa en todos los largos de contexto.

El problema importa por dos razones. Una es económica: las mitigaciones existentes consisten en re-inyectar las instrucciones en cada turno, y eso multiplica el costo en tokens a escala de miles de millones por día. El otro es la seguridad: los *guardrails* (las reglas que impiden que el modelo produzca contenido dañino) son instrucciones como cualquier otra. Si decaen, el sistema no emite ningún error: falla en silencio. Este trabajo aporta una medición, hasta donde sé inédita, de cuánto varía la retención *entre distintos tipos de instrucción*, y la justificación que esa medición ofrece para una estrategia de refuerzo selectivo.

Contexto y definición del problema

Que los LLMs olvidan ya está documentado; lo que el trabajo busca entender es *cómo* lo hacen. La literatura informa métricas agregadas: un cumplimiento medio que cae de un valor a otro. Pero un promedio oculta su estructura interna. Si cada instrucción decae a un ritmo distinto, el promedio mezcla instrucciones que el modelo retiene perfecto con otras que ignora.

Esa distinción determina qué estrategia de mitigación tiene sentido. Una instrucción que el modelo ya cumple por defecto no necesita recordatorio; una que empeora al repetirse no debería reforzarse jamás. Sin saber a qué tipo pertenece cada una, toda política de refuerzo opera a ciegas. De aquí surge la pregunta que guía el trabajo: *¿la retención de instrucciones es homogénea o heterogénea entre tipos de instrucción?* Si fuera homogénea, reforzar todo por igual bastaría; si fuera heterogénea, hay que reforzar selectivamente, y para eso primero hay que medirla.

Revisión de la literatura

Agrupo la literatura en cuatro bloques, según la función que cumplen en el argumento.

Medición del olvido.

Laban et al. (2025) compararon el mismo modelo en dos escenarios (toda la información de una vez frente a la misma tarea fragmentada en turnos) sobre quince modelos, y hallaron una caída promedio del 39% en el escenario multi-turno. Un detalle relevante: la *aptitud* del modelo baja poco, mientras que la inestabilidad sube mucho; lo que se degrada no es tanto la capacidad como la fiabilidad. He et al. (2024), en Meta, definieron el *Instruction Forgetting Ratio* y mostraron que la precisión de o1-preview cae de 87,7% a 70,7% en tres turnos. Du et al. (2025) agregaron que ampliar la ventana de contexto, por sí sola, no resuelve la degradación. Los tres informan promedios pero ninguno estima efectos por instrucción ni deriva de ahí una política de refuerzo.

Mecanismo: posición y saturación.

Liu et al. (2024) identificaron una curva en forma de U (*lost in the middle*): la información ubicada al comienzo y al final pesa más que la del medio. Por eso las del system prompt (posición cero) resisten mejor que las preferencias de mitad de conversación. Mu et al. (2025) describieron un efecto distinto, de saturación: al acumular reglas (de 1 a 20 en su prueba), la adherencia a cada una cae marcadamente. Uno es de posición; el otro, de cantidad, y juntos muestran que la adherencia no decae de modo uniforme sino estructurado.

Implicaciones de seguridad.

Anil et al. (2024) midieron empíricamente que la tasa de *jailbreak* aumenta de forma sistemática con la cantidad de ejemplos previos en el contexto (*many-shot jailbreaking*): cuanto más largo, más vulnerable. Rivasseau (2025) aportó la formalización (el cociente entre los tokens del system prompt y el contexto total tiende a cero a medida que la conversación crece) y propuso interrupciones periódicas, aún sin evaluación empírica.

Modelos bayesianos del comportamiento.

Zhang et al. (2025) mostraron que los LLM se comportan como filtros bayesianos descontados, con un factor <1 : el prior pierde peso frente a las observaciones recientes. Esto conecta con Laban et al. (2025): una varianza que explota es justo lo que produce un posterior inestable. En educación, el *Bayesian Knowledge Tracing* (Corbett & Anderson, 1994) estima desde hace tres décadas, turno a turno, la probabilidad de que un estudiante domine un concepto. La inversión conceptual que propongo es tratar al LLM como el “estudiante” y a sus instrucciones como los “conceptos”.

Comparación de enfoques

Las estrategias actuales comparten un mismo límite (Tabla 1).

Estrategias de mitigación del context rot.

Enfoque

Idea

Costo

Límite

Repetir el prompt

Re-inyectar todas las reglas cada turno

N entrada

Saturación: más reglas pueden empeorar (Mu et al., 2025)

Jerarquía de instrucciones (Wallace et al., 2024)

Priorizar por origen, aprendido en el entrenamiento

Entrenamiento

Ordena roles, no instrucciones; es estática

Duplicar el prompt (Leviathan et al., 2025)

Repetir la consulta (<Q><Q>)

2 entrada

Duplica todo por igual, sin priorizar

Recordatorios fijos (Dongre et al., 2025)

Re-inyectar el objetivo en turnos predefinidos

1,2

Calendario fijo, no adaptativo

Las cuatro hacen lo mismo: tratan al conjunto de instrucciones como un bloque y lo refuerzan por igual. Ninguna decide *qué* reforzar ni *cuándo*. La jerarquía de Wallace et al. (2024) ordena roles, pero dentro de un mismo rol no aborda el decaimiento temporal; la duplicación de Leviathan et al. (2025) y los recordatorios de Dongre et al. (2025) reinyectan el contenido completo sin distinguir entre lo que el modelo ya retuvo y lo que olvidó. Todas asumen, implícitamente, que las instrucciones son intercambiables; la medición que sigue muestra que no lo son.

Limitaciones de los enfoques actuales

Algunos trabajos empiezan a distinguir tipos: Wu et al. (2025) muestran que ciertas restricciones (de rango, periódicas) se degradan más que las de paso único en generación larga. Pero ninguno estima el efecto por tipo de instrucción ni deriva de ahí una política de refuerzo, y ese es el hueco. La métrica agregada esconde la estructura que importa: un cumplimiento medio del 70% no distingue diez reglas al 100% y diez al 40% de veinte al 70%, y esa diferencia cambia qué conviene reforzar. El refuerzo uniforme, en consecuencia, desperdicia presupuesto en instrucciones que no lo necesitan y puede ser contraproducente en aquellas que empeoran con la repetición. Además, hasta donde llega esta revisión, las grandes empresas y laboratorios

(Anthropic, OpenAI, Google, Meta) no publicaron mediciones de context rot desagregadas por tipo de instrucción sino que las mitigaciones son uniformes, en parte, porque no se cuenta con datos para discriminar.

Esta limitación motiva directamente el trabajo: sin una medición por instrucción, el refuerzo selectivo sería especular. Por eso, antes de proponerlo, había que producir la medición que lo justificara. La propuesta que sigue se deriva de cubrir ese vacío.

Propuesta de solución

La propuesta tiene dos partes. La primera mide la heterogeneidad de la retención (el grueso del trabajo y el aporte metodológico); la segunda es el sistema de refuerzo selectivo que ese resultado habilita; el sistema solo se justifica si la medición muestra esa heterogeneidad.

El *harness*: cómo medí la retención

Para medir si una instrucción se olvida hace falta una conversación realista, larga, con presión de contexto creciente, y al final una consigna que revele si el modelo todavía respeta algo que se le pidió treinta mensajes atrás. Construí en Python un *harness* con las siguientes piezas.

Codebases reales como fuente de presión. En lugar de rellenar el contexto con texto sintético, el harness opera sobre tres librerías reales del ecosistema bayesiano de Python, elegidas por tamaño creciente: Bambi (Capretto et al., 2022) (10K líneas), ArviZ (Kumar et al., 2019) (25K) y PyMC (Abril-Pla et al., 2023) (57K). Durante los primeros veinte turnos inyecta en la conversación archivos de código fuente reales de esas librerías, lo que genera un gradiente de presión (Bambi alcanza 98K tokens de contexto; PyMC, 203K) y separa el efecto del contexto del de la instrucción.

Plantado de decisiones. Definí doce “decisiones” de programación: preferencias de estilo, arquitectura, testing, nomenclatura, dependencias y documentación. En la condición *tratamiento*, esas doce preferencias se plantan como pedidos casuales, en línea, dentro de los mensajes del usuario (seis temprano, en los turnos 1–7; seis tarde,

en los 14–19); por ejemplo: “*Importante: cuando escribas operaciones con arrays en este proyecto, usá siempre broadcasting de numpy en vez de for loops*”. Son preferencias del usuario inyectadas a mitad de conversación, no reglas del system prompt: el caso difícil, sin la ventaja posicional de la posición cero. La condición *baseline* es idéntica pero no planta nada, y mide qué hace el modelo por defecto.

Turnos de evaluación. En los turnos 20–25, el usuario solicita tareas de generación de código diseñadas para forzar cada decisión. Cada turno de test evalúa exactamente dos decisiones, una temprana y una tardía, para medir el olvido reciente y también el de largo plazo.

Verificadores determinísticos. Aquí reside la reproducibilidad. Cada una de las doce decisiones se evalúa con un *checker* que analiza el código generado mediante expresiones regulares y *pattern matching*, y produce un puntaje ordinal de 0 a 3 (0 = ignorada; 3 = cumplida por completo). No interviene ningún LLM como juez en el *scoring*: el resultado es reproducible entre corridas. Tiene un límite de validez de constructo: cuando el checker no halla la señal asigna 2 por defecto, y cuatro decisiones superan el 50% de esos casos; las tres con efectos más fuertes (*parametrize*, *standalone*, *no_new*) tienen 0% de ese caso, así que los hallazgos centrales no dependen del artefacto. Por ejemplo, el verificador de *testing_parametrize* busca el decorador `@pytest.mark.parametrize` y penaliza funciones de test separadas; el de *broadcasting* detecta bucles for frente a *numpy*.

Lazo de herramientas y multiproveedor. El *runner* gestiona un ciclo de llamadas a herramientas (leer, buscar, escribir con `write_file`) sobre una capa de proveedores intercambiable, que permitió evaluar cinco modelos vía OpenRouter y la API de OpenAI. Como base, el harness usa DeepEval (Ip, 2025) para las métricas con LLM como juez (las dimensiones de persona y seguridad); en cambio, los doce verificadores de decisión que sostienen estos resultados son determinísticos y propios.

Datos y modelo bayesiano

El conjunto final consta de 28 conversaciones (9 *baseline*, 19 tratamiento) y 244 observaciones de cumplimiento (cada una es una decisión puntuada en un turno de test)

sobre cinco modelos (Qwen 3.5-27B como principal, con ambas condiciones; más Gemini 3.1 Flash Lite, GPT-5.4-nano, Nemotron 3 Super 120B y Gemma 4, solo en tratamiento).

Sobre esas observaciones ordinales ajusté una regresión logística ordinal con efectos jerárquicos por tipo de decisión. El modelo ordinal es la elección correcta porque la escala 0–3 no tiene intervalos iguales: el salto de “ignorada” (0) a “mínima” (1) no equivale al de “casi” a “completa”. Para cada observación, el cumplimiento latente $i = d[i] + d[i] \text{tratamiento}_i$ combina un intercepto por decisión d (lo que el modelo hace sin que le digan) y un efecto de tratamiento d (cuánto mejora al pedírselo). El parámetro de interés es σ , el desvío estándar grupal de los efectos: un σ grande indica que distintas instrucciones responden muy distinto a ser enunciadas: la heterogeneidad que el refuerzo selectivo necesita. El ajuste se hizo en PyMC (Abril-Pla et al., 2023) con NUTS (Hoffman & Gelman, 2014) vía nutpie (Seyboldt, 2024), más verificaciones predictivas.

Resultados

El modelo convergió de forma limpia (cero transiciones divergentes, $R^2_{0.01}$, tamaño de muestra efectivo mínimo superior a 1.400) y las verificaciones predictivas posteriores reproducen la distribución observada de puntajes. El desvío estándar grupal de los efectos de tratamiento resultó $\sigma = 2,11$ (intervalo de credibilidad del 94%: [1,06, 3,28]): la heterogeneidad existe y es grande. Los efectos individuales, en escala de *log-odds*, van de $-2,36$ a $+5,44$ (Figura 1). El posterior separa las doce decisiones en tres grupos (Tabla 2).

Efectos de tratamiento por decisión (d) con intervalo de credibilidad del 94%. Barras oscuras: el intervalo excluye el cero; rayada: efecto negativo; claras: no concluyentes.

Instrucciones que se benefician del refuerzo. Tres decisiones tienen un efecto cuyo intervalo del 94% queda por completo sobre cero. `testing_parametrize` es la más fuerte ($\sigma = 5,44$): los modelos casi nunca usan `parametrize` por su cuenta (media *baseline*: 0,00), pero retienen la preferencia cuando se la enuncian (media tratamiento: 2,79).

module_constants (=2,76) y architecture_standalone (=2,38) siguen el mismo patrón: línea de base cercana a cero, retención apreciable al pedirlo.

Una instrucción que empeora con el refuerzo. dependencies_no_new (=−2,36; $P(<0)=0,98$) es la única decisión donde enunciar la regla *reduce* el cumplimiento: la media *baseline* es 2,25 (los modelos ya evitan agregar imports), pero pedir “no agregues imports nuevos” la baja a 0,77. Es, además, la única instrucción formulada como negación, y eso hace pensar que el problema está en el marco (la negación) y no en el contenido.

Instrucciones que no requieren intervención. Las ocho restantes tienen intervalos que cruzan el cero; varias ya parten de un cumplimiento alto (broadcasting y numpy_style: 3,00), y reforzarlas malgasta tokens.

Por último, el modelo no detectó varianza apreciable atribuible a la arquitectura ni al tamaño del codebase ($\sigma_{\text{modelo}} \approx 0,4$; $\sigma_{\text{codebase}} \approx 0,3$, con intervalos que cruzan el cero); en el modelo completo, 267 transiciones divergentes llevaron a quitar esos efectos, de modo que esta comparación es indicativa y no concluyente, más aún porque la condición *baseline* se corrió en un solo modelo. Con esa salvedad, los datos son consistentes con que lo que determina la retención es la instrucción misma. El patrón se sostiene al restringir el análisis a Qwen ($r = 0,80$), con una excepción: architecture_standalone no replica en ese subconjunto (β pasa de 2,38 a −0,71), de modo que las decisiones claramente beneficiosas se reducen de tres a dos.

Prioridades de refuerzo derivadas del posterior. La ganancia es el cambio esperado en $P(\text{puntaje2})$ al reforzar.

Decisión

Ganancia

Acción

testing_parametrize

+0,78

Reforzar

module_constants

	+0,51
Reforzar	
architecture_standalone	
	+0,34
Reforzar	
naming_underscore, assert_almost, regex_comments	
	+0,14 a +0,24
Incierto	
6 decisiones restantes	
	<+0,04
Omitir	
dependencies_no_new	
	-0,47
Nunca reforzar	

El sistema que esto habilita

La medición es el prerequisite de un sistema de refuerzo selectivo basado en Bayesian Knowledge Tracing. En el harness ya está prototipado un *tracker* bayesiano que mantiene una creencia gaussiana $N(,2)$ sobre el cumplimiento latente de cada instrucción, la actualiza con cada observación (aproximación de Laplace sobre un *Graded Response Model*) y la hace decaer entre turnos (<1). Sobre esa creencia, una política puede reforzar *selectivamente* solo las instrucciones con menor probabilidad de cumplimiento, hasta un presupuesto fijo de tokens: `parametrize` recibe refuerzo, `no_new_imports` queda excluida, y las que ya se cumplen no gastan presupuesto. La Tabla 2 es, en esencia, esa regla de asignación.

Conclusiones

Este trabajo midió la retención de instrucciones por tipo en asistentes de programación basados en LLMs y encontró que es marcadamente heterogénea: los efectos del

tratamiento varían en más de un orden de magnitud. Algunas preferencias se retienen apenas se las enuncia. La mayoría ya se cumplen por defecto, así que reforzarlas solo desperdicia tokens. Y hay una que empeora cuando se la refuerza: la única formulada como negación. No se detectó varianza apreciable atribuible al modelo ni al nivel de presión de contexto, aunque el diseño limita esa comparación: los datos apuntan a que lo que decide si una instrucción se retiene es la instrucción misma.

La literatura mide el olvido en agregado y nunca por instrucción; ese es el dato que faltaba y que acá medí. La implicancia práctica es directa: el refuerzo uniforme malgasta la mayor parte de su presupuesto y puede ser contraproducente, mientras que una política derivada de la medición indica cuáles conviene reforzar y cuáles dejar sin intervención, incluida la que no hay que reforzar nunca. Las probabilidades de retención estimadas son el insumo que un sistema de Bayesian Knowledge Tracing necesitaría para funcionar.

Los comentarios de los revisores de ASAIL orientan las siguientes líneas. La más inmediata es ampliar y blindar la medición: correr la condición baseline en todos los modelos (acá quedó atada a uno) y sumar réplicas, y complementar los verificadores determinísticos con un panel de jueces LLM de otras familias, ciegos a la condición, para separar el sesgo de medición del efecto real. Una segunda línea, sugerida en la revisión, afila la negación: reformular como negación las instrucciones que sí se benefician aislaría si pesa el marco o el contenido. Quedan, además, la medición temporal densa para estimar, la evaluación en lazo cerrado frente al refuerzo uniforme, y la validación sobre conversaciones reales en lugar del banco simulado.